Modular-monolith architecture (decision)

Companion to reusable-boundary.md. That doc covers the **horizontal** framework/screen split; this one covers the **vertical** split of the business layer into modules.

> **Status: COMPLETE (2026-07-29), and this is now the description of what exists — not a plan. Steady state = modular monolith** — one process, one database, code-level modules, each its own assembly, runnable on its own when needed. All 8 business domains are promoted, `TflCbsServices` is dissolved, and Phase C shipped too (four thin per-module hosts behind a YARP gateway). This is *not* a move to microservices (see Cautions, and Why-Modular-Monolith-Not-Microservices.md for the full rationale). The staged roadmap in §6 is kept as the decision record of how it was done and why each choice was made. Last reviewed 2026-08-21.

Why

TrustBank CBS is migrating ~1,458 screens / 5,317 procs. The business layer (`TflCbsServices`) must be organized so each **module** (bounded domain) is independently owned, separately buildable, and **runnable on its own when a deployment needs only some modules**. The code is already cleanly partitioned — zero cross-module coupling, namespaces map 1:1 to folders, `Framework` is already a horizontal assembly — so this is mostly about formalizing boundaries and composition, not untangling.

The shape today (what §6 delivered)

Nine module assemblies, each namespace == assembly, each an `ICbsModule` that self-registers:

	Module	Owns
Core (`ICoreCbsModule` , always registered)	`TflCbs.Modules.General`	Menu, bank variables, broadcast/alerts, the maker-checker scroll, the process-block check — three of the four framework ports
	`TflCbs.Modules.Reference`	Geography masters (State/District/Taluka) — pure CRUD + lookup, ≥2 consumers
	`TflCbs.Core.Authentication`	Login, credentials, session validation, `LoginInfo` ; implements `ISessionValidator`
Domains (gated by `Modules:Enabled`)	`TflCbs.Modules.HR`	`DiscipActionHistoryService`
	`TflCbs.Modules.Lockers`	`LockerTypeService`

	Module	Owns
	`TflCbs.Modules.Accounts`	`BusinessAssessmentService`
	`TflCbs.Modules.Clearing`	`InwardClearingService`
	`TflCbs.Modules.Administration`	`ModuleService`, `RoleService`
	`TflCbs.Modules.RetailBanking`	`AccountService`, `HoldingAmountService` (+ `General.Contracts` for `IScrollService`)

Plus `TflCbs.Abstractions` (the floor) and `TflCbs.Modules.General.Contracts` (the only published cross-module surface). Full per-project detail: `../architecture-overview.md`.

The decisions, up front

- **No library per* service*. One assembly per* `Service` would be hundreds of micro-assemblies with

no benefit. The unit of modularity is the **module = bounded domain** (General/Bank, Lockers, RetailBanking, HR, Clearing, Accounts, Reports, Reconciliation, …); a module holds **many** services in **one** assembly.

- ***One library per* module *— staged, not big-bang. Adopt the module* pattern* first inside today's

single assembly (cheap, reversible, ~80% of the value), then **promote folders to per-module assemblies on demand**, starting with the modules you most want to run independently.

- **Steady state stays one process + one database.** "Run a module separately" is a

composition/deployment convenience (toggle which modules a host loads), not data or transaction separation.

1. Module = vertical slice (the unit)

A **module** is named `TflCbs.Modules.<Name>` and owns its full stack:

- **Services + data access** — today's `TflCbsServices/<Domain>/` folder.
- **Public surface (contract)** — the service classes (or interfaces) + their request/response models +

`Result`. Everything else trends to `internal` once the module is its own assembly.

- **Registration entry point** — `public static class <Name>Module : ICbsModule` (see §2).
- **UI** — **promoted (2026-06-30)** from a host Area to a per-domain **Razor Class Library**

`TflCbs.Modules.<Name>.Web` (controllers + views + view models + domain JS via `~/_content/`). Shared layout/partials/assets live in `TflCbs.Web.Shared`; login/logout in `TflCbs.Core.Authentication.Web`; the non-auth shell in `TflCbs.Core.Shell.Web`. The thin host composes them via gated `AddApplicationPart` keyed off `Modules:Enabled` (same list as services). **Note:** the `Bank` web module is backed by the `General` service module.

- **Framework ↔ module decoupling (2026-06-30).** Framework no longer depends on the concrete General/Auth

services. The services it needed (broadcast text, bank variables, menu rows, session re-validation) are behind **ports** in `TflCbs.Abstractions` — `IBroadcastSource`, `IBankVariableSource`, `IMenuSource`, `ISessionValidator` — implemented by `GeneralService` / `BankVariableService` / `MenuListService` / `AuthenticationService` and registered by their modules. Framework ships **no-op defaults** (`TryAdd`) for the first three and resolves `ISessionValidator` optionally, so a host runs **without** General or Authentication (degraded: empty marquee/bank-vars/menu, no session re-check). Arch test `Framework_must_consume_business_modules_only_through_abstraction_ports` locks this.

- **First Phase-B promotion (2026-07-01): `TflCbs.Core.Authentication` is its own assembly.** With the

`ISessionValidator` seam in place, `AuthenticationService` (+ its models + `AuthenticationModule`) moved out of `TflCbsServices` into a peer data-layer assembly `TflCbs.Core.Authentication` (namespaces `TflCbs.Core.Authentication[.Models]`). **Module discovery is now multi-assembly:** `AddCbsModules(config, params Assembly[] extraModuleAssemblies)` scans `TflCbsServices` **plus** any promoted module assemblies the host passes — the host calls `AddCbsModules(config, typeof(AuthenticationModule).Assembly)`. This is the template for every future folder→assembly promotion: move the folder, give it `TflCbs.<Area>` namespaces, add its assembly to the host's `AddCbsModules` hand-off. The one cross-module call the split surfaced — `AuthenticationService` reusing General's process-block check — goes through the **`IProcessBlockCheck` contract** (`General.Contracts`, second published contract after `IScrollService`); `GeneralGrp11Service` implements it, the General module registers it, and `AuthenticationService` injects it. So Authentication depends on General only via `.Contracts` (arch test `Authentication_may_call_General_only_through_its_Contracts`), and a host that enables `Authentication` must also enable `General` (a legitimate module→module dependency).

- **Tests + Demo** — per-module fixtures mirroring the existing `TflCbs.Tests` / `.Demo`

convention, kept in sync by the **cbs-sync-test-demo** agent.

Granularity rule: one assembly per module — never per service, never per screen. Add a separate `TflCbs.Modules.<Name>.Contracts` assembly only when another module actually needs to call this one (today: zero such calls — defer it).

2. Module composition pattern (`ICbsModule`) — the key enabler

A tiny contract every module implements; the host composes modules from configuration:

```
public interface ICbsModule {
    string Name { get; }
    void AddServices(IServiceCollection services, IConfiguration config); // DI for this module
    // (later, when modules carry their own endpoints/areas) void MapModule(IEndpointRouteBuilder
}
```

- Each module ships `<Name>Module : ICbsModule` whose `AddServices` registers its services — replacing

the hand-written `builder.Services.AddScoped<XService>()` lines in `Program.cs`.

- The host reads an **enabled-modules list** and loads only those:

``jsonc "Modules": { "Enabled": ["General", "Lockers", "RetailBanking"] } // or "" *for all* ` ` `csharp builder.Services.AddCbsFramework(builder.Configuration); // Discovery is FULLY EXPLICIT since TflCbsServices was dissolved: every host names every // module assembly it deploys. There is no implicit anchor assembly to scan any more. builder.Services.AddCbsModules(builder.Configuration, typeof(GeneralModule).Assembly, typeof(ReferenceModule).Assembly, / … */);``

- **This is how a module "runs separately" with zero extra projects:** the same host binary runs the

full bank (`"*"`) or a single module (`["Lockers"]`) by config alone. Recommended default.

- **Deployed-module cross-check.** `AddCbsModules` scans the output directory at startup and throws if

a `TflCbs.Modules.<X>` / `TflCbs.Core.<X>` assembly is deployed but was not passed in. Nothing else catches that: a host on `""` *never names a module so the unknown-name check cannot fire, and a missing* `ICoreCbsModule` *degrades* ***silently*** *to the framework's no-op ports (empty menu). Composition* tests* that build partial sets opt out with `verifyDeployedModules: false`.

3. "Run modules separately" — two strategies (use the lightest that fits)

- **Strategy 1 — one configurable host (default).** `TflCbs.Host.Main` loads the modules in

`Modules:Enabled`. A focused deployment = same binary, different config. No project proliferation.

- **Strategy 2 — thin per-module host (only when truly needed).** A ~20-line `TflCbs.Host.<Name>`:

`AddCbsFramework().AddServices(<Name>Module)` + that module's RCL. Justified only when a module needs its **own deploy cadence / scaling / process** (e.g. a heavy Reports or Reconciliation runner). Same `ICbsModule` contract — no redesign.

Both share the **horizontal foundation** (`TflCbs.Framework` + `TflCbs.Abstractions` + `TflCbs.Entities` + `TflOmniDb`); only the *module set* differs.

4. Shared foundation & boundaries

Horizontal assemblies (single, shared — do NOT split per module):

- `TflCbs.Framework` — web plumbing (session, search, RowToken, menu, access guard,

`DatabaseSelector`). Already its own assembly; already barred from depending on screens.

- **`TflCbs.Abstractions` — extracted (2026-07-28).** The shared foundation every module and the framework

compile against, in one namespace `TflCbs.Abstractions`: `Result` / `Result<T>`, `ICbsModule` / `ICoreCbsModule`, the four framework ports (`IBroadcastSource`, `IBankVariableSource`, `IMenuSource`, `ISessionValidator`), and the two DTOs that cross the framework boundary (`MenuListRow` — the `IMenuSource` payload; `BranchListItem` — surfaced on `ModulesIndexViewModel`). **Deliberately dependency-free**: no `TflCbs.Entities`, no `TflOmniDb`, no ASP.NET, no ADO.NET drivers. Everything above it references this, so keep it that way. - Superseded the old `TflCbs.Services.{Common,Abstractions,Composition}` namespaces (single sweep, 256 files). - **`ModuleRegistration.cs` (`AddCbsModules`) deliberately stayed in `TflCbsServices`**: `DiscoverModules` anchors on `typeof(CbsModuleServiceCollectionExtensions).Assembly` to mean "the monolith services assembly". Moving it would silently scan the wrong assembly and register nothing. It moves only once `TflCbsServices` is empty — at which point discovery must become fully explicit (every host passing every module assembly). Its `Microsoft.Extensions.DependencyInjection` namespace means no host call site changed. - Still to fold in (unchanged from the original plan): the duplicated `IsDuplicate` / `IsForeignKey` exception helpers (repeated in 8+ services), the repeated paging/sort helpers, shared base search contracts.

- **`TflCbs.Modules.General.Contracts` — extracted (2026-07-28).** General's published cross-module surface:

`IScrollService` + `ScrollSources` (consumed by RetailBanking) and `IProcessBlockCheck` (consumed by `TflCbs.Core.Authentication`). Carries the `TflOmniDb` bare-DLL reference because `IScrollService` takes `IUnitOfWork`. This is what let Authentication drop its `TflCbsServices` reference — see §6.

- `TflCbs.Entities` — stays **one** assembly. Entities are cross-cutting, auto-generated, and shared

(e.g. `G_STATE` used by District + Taluka). Splitting per module would create cross-module entity references — avoid.

- `TflOmniDb` — the multi-provider DAL. Still consumed as a **bare DLL by** `HintPath`, but since

2026-08-19 its source is vendored into the repo at `libs/TflOmniDb/` (as is `TflSecurityCrypto` at `libs/TflSecurityCrypto/`), so a fresh clone builds without a folder outside the solution root. The DLL-by-HintPath style is kept deliberately: it stops the library's transitive package graph leaking into every module, which is why each project in the closure re-declares its ADO.NET drivers.

Allowed dependency directions:

- module → `TflCbs.Abstractions`, `TflCbs.Entities`, `TflOmniDb`, `TflOmniLog` ✅
- module → `Framework` ❌ — **the arrow does not exist in either direction.** A service module has

zero ASP.NET (abstractions-only DI/Configuration packages, arch-tested); it never sees the framework, and the framework never sees it. Where the framework needs business data it declares a **port** in `TflCbs.Abstractions` and a core module implements it.

- module A → module B ❌ (only via B's `.Contracts` interface through DI, if ever needed)
- `Framework` → any module ❌ (Framework stays horizontal)
- host → `Framework` + chosen modules + their web RCLs ✅

Enforcement — ✅ built. `TflCbs.ArchTests` (NetArchTest) carries the inter-module rules, and now that every module is its own assembly they are compiler-enforced (no project reference to add), like the Framework⊥screens boundary. The suite's remaining job is the meta-guard below: noticing when a rule's *level* changes.

`Framework → any module` **is now compiler-enforced (2026-07-28).** `TflCbs.Framework` dropped its `TflCbsServices` project reference and links only `TflCbs.Abstractions`, `TflOmniLog` and `TflOmniDb`. The existing port guard (`Framework_must_consume_business_modules_only_through_abstraction_ports`) is a *denylist* of named concrete types, so it is backed by `Framework_must_not_reference_any_business_module_assembly`, which asserts on `GetReferencedAssemblies()` — re-adding the reference fails loudly and says why.

Arch-rule enforcement levels, and why they are tracked (2026-07-28)

A NetArchTest namespace rule can only fail if the forbidden types are **reachable** from the inspected assembly. When a project reference goes away the rule keeps passing — but as a *tautology*: the dependency became impossible, so nothing is being checked, and green silently stops meaning "the code is clean". Extracting `TflCbs.Abstractions` did exactly that to two rules in one commit.

`TflCbs.ArchTests` therefore declares every boundary in one registry (`ArchBoundaries`) with an explicit level, and `BoundaryGuaranteeTests` re-verifies the level against the live reference graph:

Level	Meaning	What the meta-guard asserts
`Enforcement.Namespace`	Targets ARE reachable; the namespace rule is the only thing stopping the dependency	every target still reachable — else the rule is a tautology and must be retargeted
`Enforcement.Reference`	Targets are NOT reachable; the missing project reference is the guarantee	nothing became reachable — else a reference was re-added and the boundary is lost

Consequences worth knowing:

- `RetailBanking → General` **is the only namespace rule with real teeth today** (both share `TflCbsServices`).

Promoting General flips it to `Reference`, and the meta-guard will go red asking for exactly that change.

- `General` **is core, not a sibling.** It is `ICoreCbsModule`, always registered, and owns the shared

masters — the HR menu hosting General's State/District/Taluka screens is the *documented* legacy-placement decision, not a leak. General must never appear in a sibling ban list.

- **The live cross-domain leak path is web → sibling* service*, not web → sibling* screen.* Every web RCL

references the whole `TflCbsServices` assembly, so `Hr.Web` can inject `LockerTypeService`; sibling *screens* live in an unreferenced RCL and were never reachable. `Domain_web_module_must_not_use_a_sibling_domains_services` closes the former.

- **Deploy config is checked against the code.** `ModuleConfigTests` asserts every name in

`Modules:Enabled` across `compose.yaml` / `appsettings.json` resolves to a registered `ICbsModule`, because `AddCbsModules` throws on unknown names. A stale `Misc` entry had been crashing all four Docker thin hosts at startup; nothing caught it because nothing compared config to code.

5. Enabling infrastructure (done BEFORE splitting — it made the split bearable)

The old no-`.sln` + bare-DLL setup did not scale to 15+ module projects. The prerequisites, and where each landed:

1. **`Directory.Build.props` at the root — ✅ done.** Defines `TargetFramework net10.0`, `Nullable`,

`ImplicitUsings` and `ManagePackageVersionsCentrally` once, plus the publish guard that keeps `appsettings.Development.json` out of published artifacts. It deliberately does **not** inject the `TflOmniDb` / `TflCbs.Entities` bare-DLL references or the ADO.NET drivers globally — those belong only to projects whose closure needs them. Versions live in `Directory.Packages.props` (Central Package Management), so csproj `PackageReference`s are version-less. **Highest-leverage step, as predicted.**

1. **A real solution — ✅ done, as `TflCbsNet10Sol.slnx`** (36 projects). The `TflCbs.Entities` build-order

problem was **not** solved by making it a plain `ProjectReference`: the generated-DLL contract requires the HintPath. It was solved on **2026-08-21** by giving all 10 consumers a `ProjectReference … ReferenceOutputAssembly="false"` **build-order edge** — MSBuild gets the ordering, the compiler still binds the DLL. A parallel Rebuild All previously raced and failed with `CS0246`; the manual "build `TflCbs.Entities` first" discipline is now gone.

1. **Solution filters (`.slnf`) per module group — ⛔ not built, and superseded.** The need it was

meant to serve (a team opening just their module + foundation) is met better by the `codebase_shared/` pattern: a self-contained handover tree holding only the projects that team edits, with the rest of the platform as prebuilt DLLs in `refs/` — which is why nine assemblies now set `GenerateDocumentationFile`, so IntelliSense carries the API semantics where no source does.

6. Phased roadmap (incremental, reversible at each step)

- **Phase A — modular pattern inside the current assembly (no splits).** Add `ICbsModule` + per-module

`AddServices` / `AddCbsModules(config)`; move the `AddScoped` lines out of `Program.cs`; extract `TflCbs.Abstractions`; add NetArchTest inter-module rules; add `Directory.Build.props` + `.sln`. Delivers modules + "toggle which run" without splitting a single assembly.

- **Phase B — promote folders to per-module assemblies, on demand.** Starting with the modules you most

want to run independently, move `TflCbsServices/<Domain>/` → `TflCbs.Modules.<Name>`, flip internals to `internal`, expose via the module's registration + `.Contracts` if cross-module calls exist. One module at a time; build stays green. - **Namespaces follow the assembly** (decided 2026-07-28, and already applied to the whole web layer): a promoted domain becomes `namespace TflCbs.Modules.<Name>`, matching `TflCbs.Core.Authentication` and `TflCbs.Modules.General.Contracts`. Renaming is cheapest now and grows with every migrated screen. It costs a `using` sweep across that domain's tests, `TflCbs.Demo/Program.cs`, its Web RCL controllers, and the type-name string literals in `ArchTests/ArchBoundaries.cs`. `NamespaceDisciplineTests`

enforces the rule (namespace starts with assembly name; no namespace shared by two assemblies) — add the new assembly to its anchor table when you promote. - **`promote.ps1` rewrites `TflCbs.Services.Lab.<X>` → `TflCbs.Services.<X>`.** That rule must be updated per domain as each one is promoted, or Lab→prod promotion will land the wrong namespace. - **The foundation is in place (2026-07-28)** — §4. A promoted module now references `TflCbs.Abstractions` (+ another module's `.Contracts` if it genuinely calls one) and **never `TflCbsServices`**. `TflCbs.Core.Authentication` is the worked template: it dropped its `TflCbsServices` reference entirely, so the promotion is real rather than cosmetic. - **First domain promotion: `Lockers` (2026-07-29).** `TflCbsServices/Lockers/` → `TflCbs.Modules.Lockers` (namespace `TflCbs.Modules.Lockers`), referencing only `TflCbs.Abstractions`. Chosen over HR because it yields the **first fully isolated vertical slice**: `TflCbs.Modules.Lockers.Web` has a single controller, so it dropped `TflCbsServices` too and now links only its own service module + the horizontal layer. Measured, not assumed — `TflCbs.Modules.Lockers.dll` is present in `TflCbs.Host.Lockers` / `TflCbs.Host.Main` output and **absent** from the Hr/RetailBanking/Administration host outputs. That absence is the payoff. - **Only hosts that enable a module reference it.** Adding the reference to every host "for consistency" would re-ship the code everywhere and defeat the exercise. - *`TflCbs.Host.Main` runs `Modules:Enabled = "`**`"`**, which skips the unknown-name check* — *so a promoted module missing from* its* `AddCbsModules` call is dropped **silently** (a 500 when a screen resolves its service), whereas a thin host fails loudly at startup. Always add the assembly there. - Two arch tests had borrowed Lockers as a convenience fixture for subjects that were not Lockers (`EnabledConfig("Lockers")`, `typeof(LockerTypeService)`); both were re-pointed at `Administration`, a domain still inside the monolith, so they stay single-assembly and keep testing config gating.

- **Second domain promotion: `HR` (2026-07-29).** `TflCbsServices/HR/` → `TflCbs.Modules.HR` (namespace `TflCbs.Modules.HR`), referencing only `TflCbs.Abstractions`. Named `.HR` because the module's registered name is `"HR"`; that casing is now consistent across the module class, `Modules:Enabled` in every host and `compose.multi.yaml` (they previously reconciled only through case-insensitive matching). The web RCL stays `TflCbs.Modules.Hr.Web` and the area/routes stay `/Hr/…` — assembly and URL identifiers are a separate axis. - **Unlike Lockers, `Hr.Web` keeps its `TflCbsServices` reference** — its District/State/Taluka screens are backed by the core General module and will be until General is promoted. So the payoff here is narrower: `TflCbs.Modules.HR.dll` no longer ships to the Lockers/RetailBanking/Administration hosts, but the RCL is not yet monolith-free. - HR reads General-**owned tables** (`G_TITLE`, `G_DESIGNATION`, `G_ORGELEMENTDEPARTMENT`) through `TflCbs.Entities`. That is table-level coupling to the shared schema, not an assembly dependency — it needs no `.Contracts` and does not make HR a non-leaf.

- **Remaining leaves promoted (2026-07-29): `Accounts`, `Clearing`, `Administration`, `RetailBanking`.** All four were true leaves; `RetailBanking`'s single cross-module edge (`IScrollService`) already went through `TflCbs.Modules.General.Contracts`, so it references that and nothing else from the monolith. **`TflCbsServices` now contains only `General` and `Composition`.** - `Administration.Web` and `RetailBanking.Web` each had exactly one service dependency — their own domain — so both dropped `TflCbsServices` too. Four of five RCLs are now monolith-free; only `Hr.Web` still links it, for the General-backed District/State/Taluka screens. - **Each thin host now ships exactly one domain module — its own.** Measured on the build output, not assumed. - Two guard changes this forced: the registry's promoted list is now keyed by **domain, not web area** (`Accounts` and `Clearing` have no RCL, so a web-area-keyed list could not express them), and `RetailBanking -/-> sibling service impls` flipped from `Namespace` to `Reference` — it was the suite's last namespace rule with real teeth, and the promotion made it a compiler guarantee.

- **Final promotion — `General` split into `General` + `Reference` (2026-07-29). `TflCbsServices` is gone.** General had been doing two jobs: core platform (menu, bank variables, broadcast, alerts, the scroll and process-block mechanisms — three of the four framework ports) and bank-wide master data. The geography masters moved to `TflCbs.Modules.Reference`; what stayed is platform. - Both are `ICoreCbsModule` — registered unconditionally. Reference data is needed everywhere (66 entity tables carry `STATEID` / `DISTRICTID` / `TALUKAID`), so gating it would mean a thin host that forgot to list it fails at runtime on a district lookup. - **Admission rule for Reference:** a master belongs there only if it has no domain behaviour (pure CRUD + lookup) **and** two or more domains consume it. Geography passes; `LockerMake` does not — Lockers-only, stays put. Without this test Reference becomes the dumping ground for anything nobody wants to place. - `Hr.Web` needed only State/District/Taluka, so it now references **Reference**, not General — making it monolith-free. **All five RCLs are now monolith-free.** - `AddCbsModules` **moved to** `TflCbs.Abstractions` **and discovery is fully explicit.** It used to start from the `TflCbsServices` assembly implicitly and add promoted assemblies to it; that assembly no longer exists, so every host must now name every module. The deployment cross-check added the same day is what makes that safe — miss one and startup aborts naming it. - `NamespaceDisciplineTests` now holds with **no exceptions**: `TflCbsServices` (assembly `TflCbsServices`, namespaces `TflCbs.Services.*`) was the last mismatch. - Follow-up **closed (2026-07-29)**: `GeneralService.GetDistrictAsync` (+ `DistrictRow` / `DistrictListResult`) was deleted rather than relocated. It had zero callers and was a stored-procedure wrapper superseded by the entity-based `DistrictService.SearchDistrictsAsync`. The **proc** `pr_g_getDistrict` is untouched: its one remaining caller is the legacy screen `HO\Bank\g_District.aspx`, so dropping it is a cutover decision, not a code one. Worth generalising — during the strangler-fig migration both apps share one database, so **"no C# callers" never means "unused"** for a proc; deleting C# is a code question, deleting a proc is a cutover question.

Guard model for promotions: `PromotedModules`

`ArchBoundaries.PromotedModules` names the domains that have left `TflCbsServices`, and every sibling rule is emitted **twice** off it: `"<X> -/-> sibling services (monolith)"` at `Enforcement.Namespace` (siblings still in the shared assembly are reachable, so only a namespace rule stops a call) and `"<X> -/-> promoted sibling modules"` at `Enforcement.Reference` (a promoted sibling is unreachable unless someone adds a project reference, so the compiler is the guard). **Each future promotion is one entry added to** `PromotedModules` — the rules re-partition themselves, and `BoundaryGuaranteeTests` re-verifies both levels against the live reference graph.

`PromotedModules` **and** `MonolithFreeWebModules` **were deliberately separate lists — and the second one is now deleted (2026-08-19).** Promoting a domain's *service* module said nothing about whether its *web* RCL still needed `TflCbsServices`: `Lockers.Web` was decoupled by its promotion (one controller, one dependency), but `Hr.Web` still linked the monolith for its General-backed screens. Conflating the two was not a hypothetical — the HR promotion initially declared `Hr.Web`'s sibling rule `Reference` on that assumption and `BoundaryGuaranteeTests` failed it immediately, reporting that `Hr.Web` could still reach `TflCbs.Services.RetailBanking` and `…Administration`. The meta-guard earning its keep on the first promotion after it was written.

Once **all** domains were promoted the distinction had no members left to express: `MonolithFreeWebModules` was a four-name list feeding exactly one rule (`"{key}.Web -/-> sibling services (monolith)"`), guarded by `if (monolithSiblings.Length > 0)` where `monolithSiblings = SiblingServiceNamespacesFor(key).Except(PromotedModuleNamespaces)`. With every domain in

`PromotedModules` that set is always empty, so the branch was dead — a list and 22 lines of doc comment describing a state that can no longer occur. It was deleted; `PromotedModules` is now the single registry, and every sibling rule is `Enforcement.Reference`.

- **Phase C — independent hosting — ✅ shipped.** Every domain's Area became an RCL, and four thin

hosts exist: `TflCbs.Host.Hr`, `.Lockers`, `.RetailBanking`, `.Administration`, fronted by `TflCbs.Gateway` (YARP) so all five hosts share one external origin — the precondition for cross-host SSO. The trigger was not scale but **proof**: each thin host's build output was measured to contain exactly one domain module, which is the evidence that the boundaries are real. The platform therefore runs in four deployment shapes from one codebase (IIS single, IIS multi-host, Docker single, Docker multi-host); see `../port-map.md` and `../single-sign-on.txt`.

Promotion triggers (folder → own assembly): module must run/deploy on its own ✅ · a team needs hard ownership + internal hiding ✅ · monolith build time hurts · module ships as a package · cross-module calls must be constrained to a contract. Promote when a trigger fires — not all modules at once.

7. Cautions

- **One database, one process is the steady state.** Core-banking flows (day-end, GL posting, clearing,

interest) span modules in a **single in-process transaction** (`DataAccess.ExecuteInTransactionAsync`). Splitting the *database*, or forcing modules into separate *processes for normal operation*, turns these into distributed transactions / sagas — high correctness and compliance risk. Module = **code** boundary; do not let "run separately" drift into microservices.

- **Process separation ≠ data separation.** A per-module host (Strategy 2) still hits the shared DB via

the same `DatabaseSelector`. That's intended. Data isolation is a far bigger, separate decision — out of scope here.

- **Entities stay shared.** Do not split `TflCbs.Entities` per module (shared tables, generated contract).
- **Contracts before cross-calls.** The first time module A needs module B, introduce B's `.Contracts`

interface and call through DI — never reference B's implementation assembly directly.